Memória Descritiva

Blue Garden

Repositório de Ficheiros Distribuído On-Line

1.ª entrega

Francisco Lourinho - 202301001

David Bação - 20230331

Lucas Nicolau - 20241526

Grupo 3

Projeto de Desenvolvimento de Software · Licenciatura em Engenharia Informática

3.º ano · 5.º semestre · 2026/2027 · IADE - Universidade Europeia

Docente: Rui Pascoal

Outubro de 2026

Resumo

O Blue Garden é um repositório de ficheiros on-line, semelhante ao Google Drive, que guarda os ficheiros de forma distribuída e os organiza automaticamente por categoria com Inteligência Artificial. Corresponde ao Trabalho 3 do briefing e junta as unidades curriculares do 5.º semestre: Engenharia de Software, Sistemas Distribuídos, Inteligência Artificial, Segurança Informática e Projeto de Desenvolvimento de Software.

A ideia partiu de uma necessidade do grupo. Acumulamos PDFs das aulas, trabalhos, faturas, fotografias e capturas de ecrã, tudo no mesmo sítio e com nomes que não dizem nada. Os serviços que existem guardam bem os ficheiros, mas a organização continua a ser feita à mão e a maioria das pessoas acaba por desistir.

No Blue Garden, o utilizador carrega os ficheiros e encontra-os já arrumados em sete categorias: Académico, Faturas e finanças, Documentos pessoais, Fotografias, Capturas de ecrã, Código e Por rever. Cada decisão tem um grau de confiança e, quando é baixo, o ficheiro vai para «Por rever». O utilizador pode sempre corrigir a categoria.

A IA usa regras simples para fotografias, capturas de ecrã e código, e um classificador (TF-IDF com Regressão Logística, comparada com Naive Bayes) para os documentos. As correções do utilizador ficam guardadas para melhorar o modelo. Toda a análise é feita nos nossos servidores, sem serviços externos.

O sistema foi pensado para continuar a funcionar quando um componente falha. Um Nginx distribui os pedidos por três backends em Python (FastAPI), os metadados ficam num replica set de MongoDB com três nós e os ficheiros, cifrados, num cluster MinIO com quatro nós. Tudo corre em Docker.

Nesta primeira fase definimos o problema, os requisitos, os diagramas UML, os mockups, a arquitetura, o modelo de faltas, a componente de IA e a segurança. As próximas fases são dedicadas à implementação, aos testes de falhas e à versão final.

1. Contexto/ Introdução

1.1. Problema abordado

Os ficheiros acumulam-se sem ordem, com nomes como «Documento1.docx» ou «trabalho final v3 (2).pdf». A extensão também não ajuda: um PDF tanto pode ser uma fatura como uns apontamentos.

Organizar à mão demora e acaba por ser abandonado. Além disso, um serviço de armazenamento tem de estar sempre disponível e não pode perder ficheiros quando uma parte do sistema falha.

1.2. Motivação

Todos no grupo já perdemos tempo à procura de um ficheiro que sabíamos que existia. O Google Drive, o Dropbox e o OneDrive guardam bem os ficheiros, mas nenhum arruma os documentos pelo conteúdo.

Público-alvo: estudantes do ensino superior (18 a 27 anos) e adultos que precisam de manter faturas e documentos pessoais organizados, sem tempo para os arrumar à mão.

1.3. Objetivos

- Guardar, organizar e pesquisar ficheiros on-line, com reciclagem.
- Categorizar cada ficheiro automaticamente, com um grau de confiança.
- Continuar a funcionar se falhar um backend, um nó de armazenamento ou um nó da base de dados.
- Proteger os dados com HTTPS, hash nas passwords e ficheiros cifrados.

2. Processo

2.1. Metodologia utilizada

Metodologia ágil, com sprints de uma semana, um quadro de tarefas (a fazer, em curso, feito) e uma reunião semanal curta. O código fica no GitHub e passa por pull request antes de entrar no projeto.

- **Fase 1** (entrega a 11/10/2026): requisitos, UML, mockups, arquitetura e modelo de faltas.
- **Fase 2** (entrega a 15/11/2026): infraestrutura, funcionalidades base e primeira versão do classificador.
- **Fase 3** (entrega a 20/12/2026): integração, testes de falhas e versão final.

2.2. Ferramentas utilizadas

- **GitHub:** controlo de versões e pull requests.
- **Quadro de tarefas:** organização dos sprints.
- **Chat de grupo:** comunicação do dia a dia.
- **Docker Compose:** arranque de todos os serviços com um só comando.
- **Ferramentas de diagramas e mockups:** UML, arquitetura e interfaces.

2.3. Tecnologias utilizadas

- **Frontend:** HTML e JavaScript.
- **Nginx:** HTTPS e distribuição dos pedidos.
- **Backend:** Python com FastAPI (três instâncias).
- **MongoDB:** replica set de três nós para os metadados e a fila da IA.
- **MinIO:** quatro nós com erasure coding para os ficheiros.
- **IA:** scikit-learn (TF-IDF, Regressão Logística e Naive Bayes).
- **Segurança:** Fernet (AES), bcrypt e JWT.

2.4. Estrutura da equipa

Três elementos: Francisco Lourinho, David Bação e Lucas Nicolau. As tarefas são distribuídas em cada sprint e o código de cada um é revisto por outro elemento.

3. Resultados

3.1. Descrição da solução desenvolvida

O utilizador entra no site, carrega os ficheiros e encontra-os já arrumados por categoria (Figura 7).

Os pedidos passam pelo Nginx, que os distribui por três backends stateless. Os metadados ficam no MongoDB (três nós) e os ficheiros, cifrados, no MinIO (quatro nós). Assim, a falha de um componente não faz perder ficheiros (Figura 1).

Cada ficheiro novo entra numa fila. O worker de IA aplica as regras ou o classificador e guarda a categoria com o grau de confiança. Abaixo de 0,6, o ficheiro vai para «Por rever». Como corre em segundo plano, o upload não fica à espera da IA.

3.2. Funcionalidades principais

- Conta e login.
- Upload e download de ficheiros.
- Pastas: criar, renomear, apagar e mover ficheiros.
- Reciclagem com recuperação.
- Pesquisa por nome.
- Categoria automática com grau de confiança.
- Correção manual e filtro por categoria.

Lista completa de requisitos nos Anexos A e B.

3.3. Contributos relevantes

- Organização pelo conteúdo, sem o utilizador criar regras ou pastas.
- Armazenamento tolerante a faltas, com ficheiros cifrados e análise feita nos nossos servidores.
- Uma IA honesta: quando não tem certeza, diz isso em vez de arrumar mal o ficheiro.

4. Reflexão/ Conclusões

4.1. Lições aprendidas

- Pensar no modelo de faltas cedo mudou o desenho, por exemplo ao pôr a fila da IA na base de dados.
- Nem tudo precisa de machine learning: regras simples resolvem fotografias, capturas e código.
- A segurança tem custos: cifrar afeta o desempenho e obriga a cuidar bem da chave.

4.2. Limitações

- O Nginx é um ponto único de falha.
- Só consideramos falhas por paragem (crash).
- Há uma só chave de cifra: se se perder, os ficheiros ficam ilegíveis.
- O dataset é pequeno e feito pelo grupo.
- Ficheiros sem texto nem metadados são difíceis de classificar.

4.3. Trabalho futuro

Na próxima fase vamos implementar a infraestrutura, as funcionalidades base e o primeiro classificador. Mais à frente, queremos:

- Dois Nginx com keepalived, para eliminar o ponto único de falha.
- Várias categorias por ficheiro.
- Deteção de ficheiros duplicados.
- Partilha de ficheiros entre utilizadores.

5. Imagens

Conceitos, modelação e mockups produzidos na primeira fase.

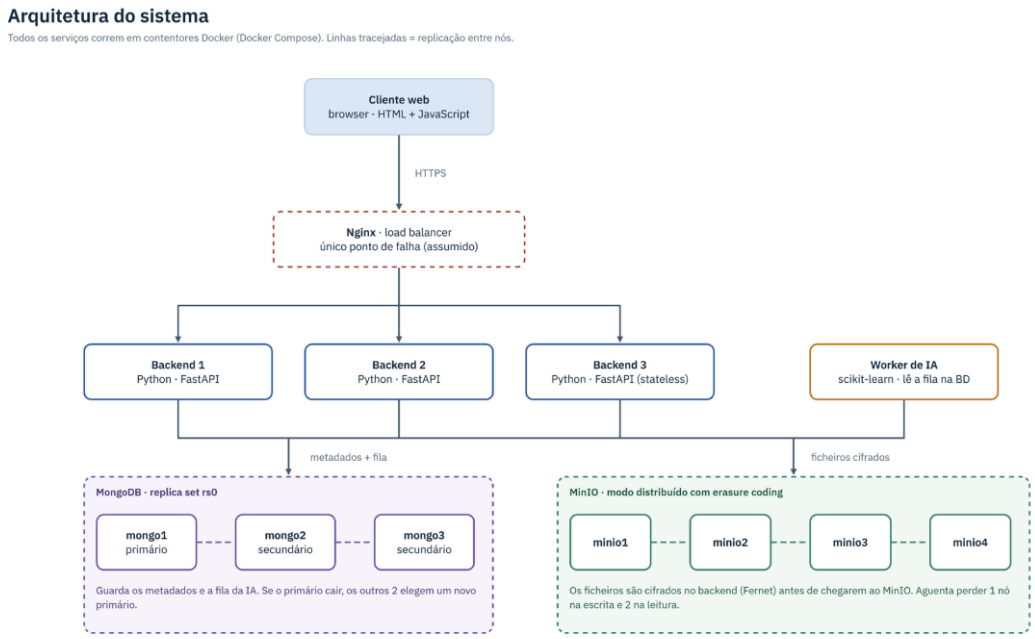


Figura 1. Arquitetura distribuída do Blue Garden

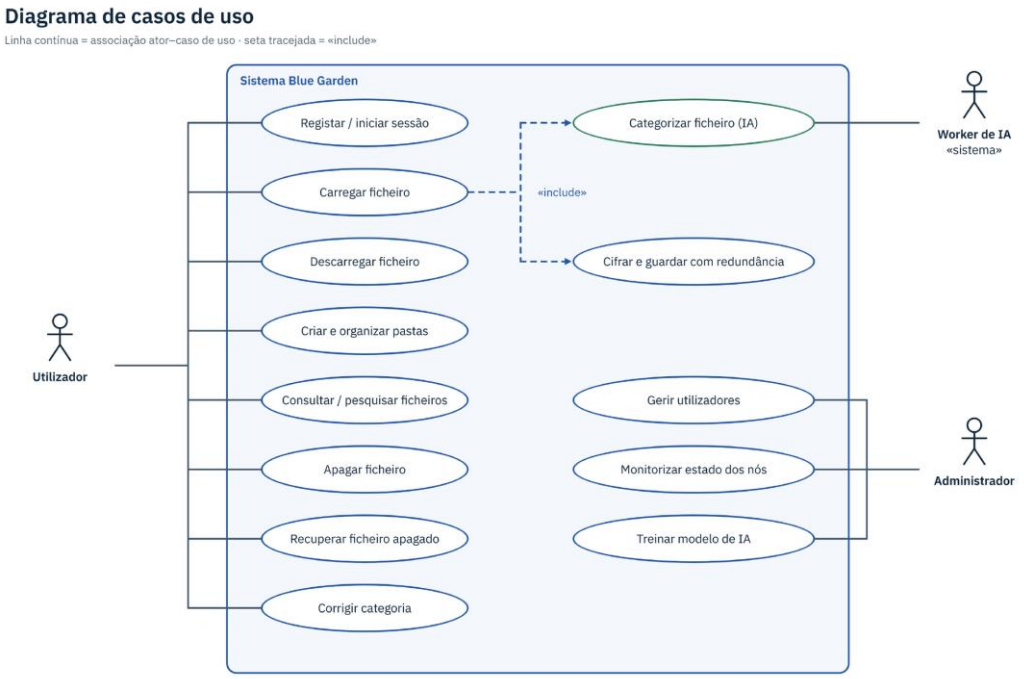


Figura 2. Diagrama de casos de uso

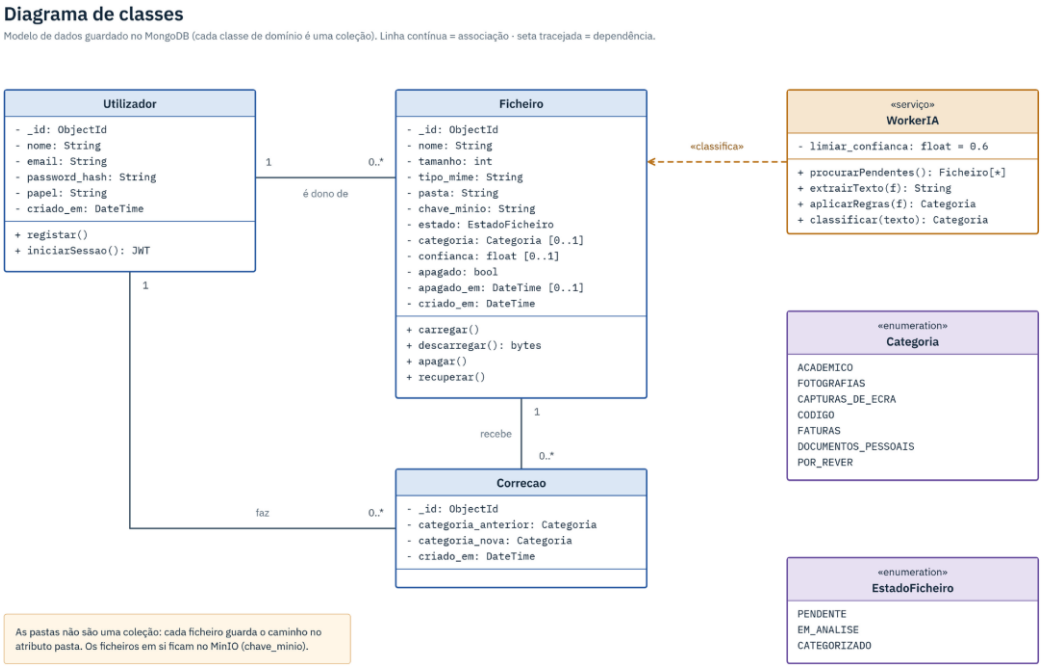


Figura 3. Diagrama de classes (estrutura dos dados no MongoDB)

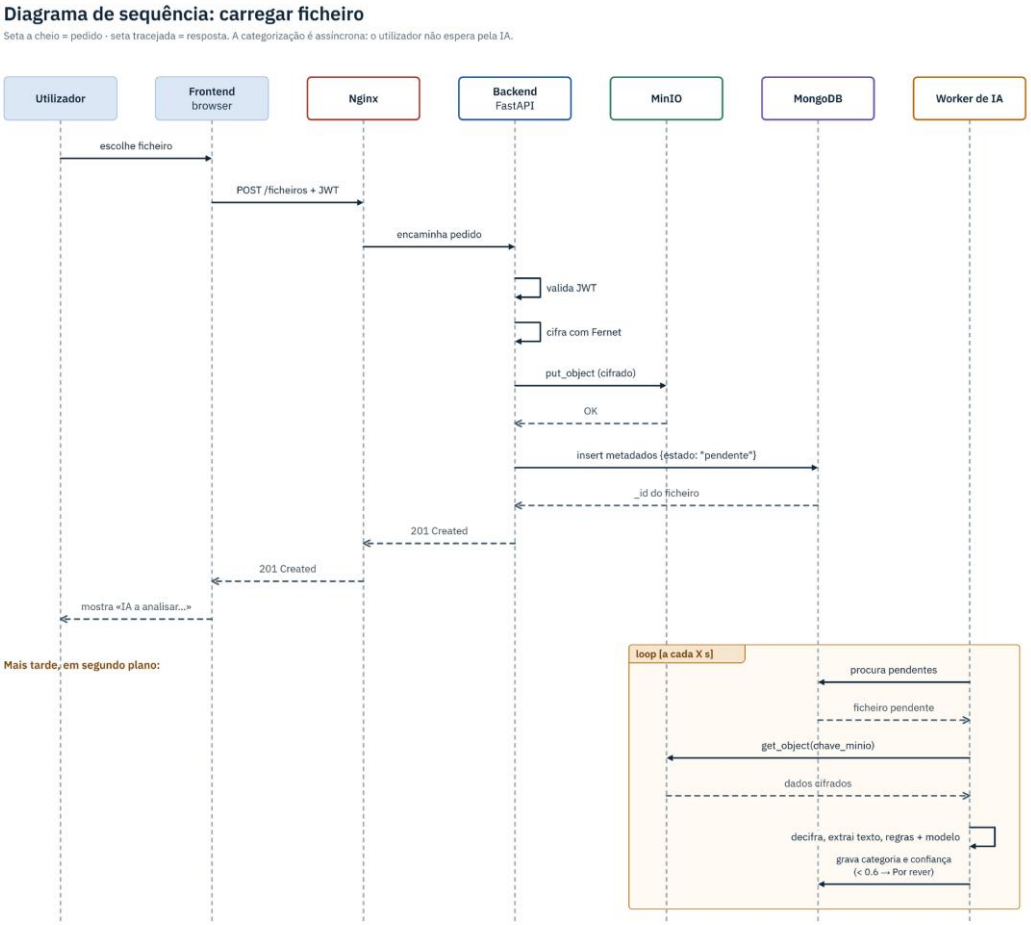


Figura 4. Diagrama de sequência do carregamento de um ficheiro

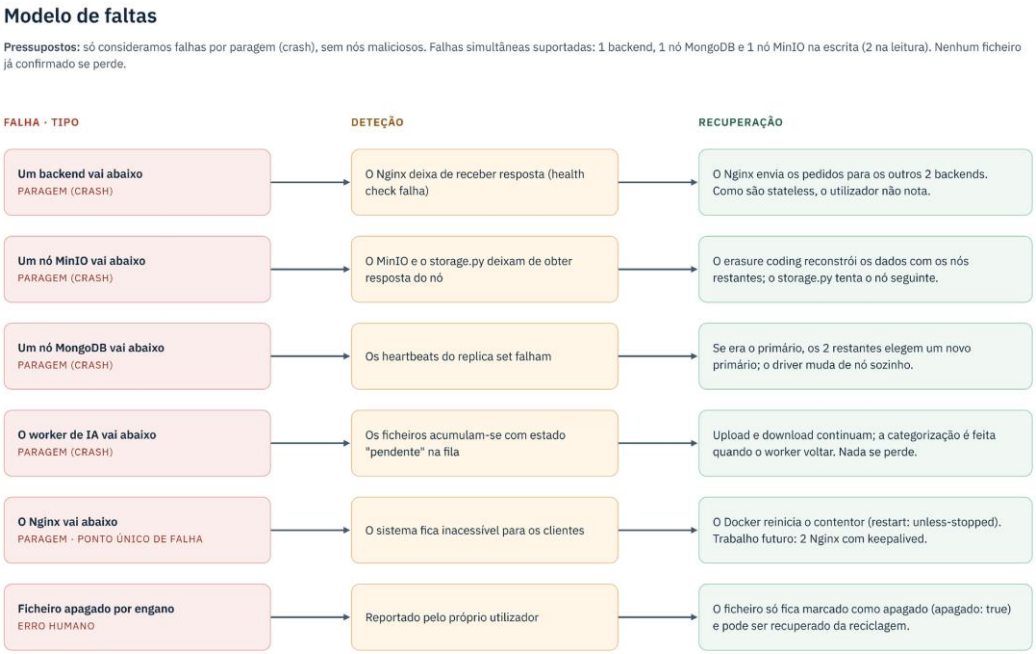


Figura 5. Modelo de faltas



Figura 6. Mockup da página de início de sessão

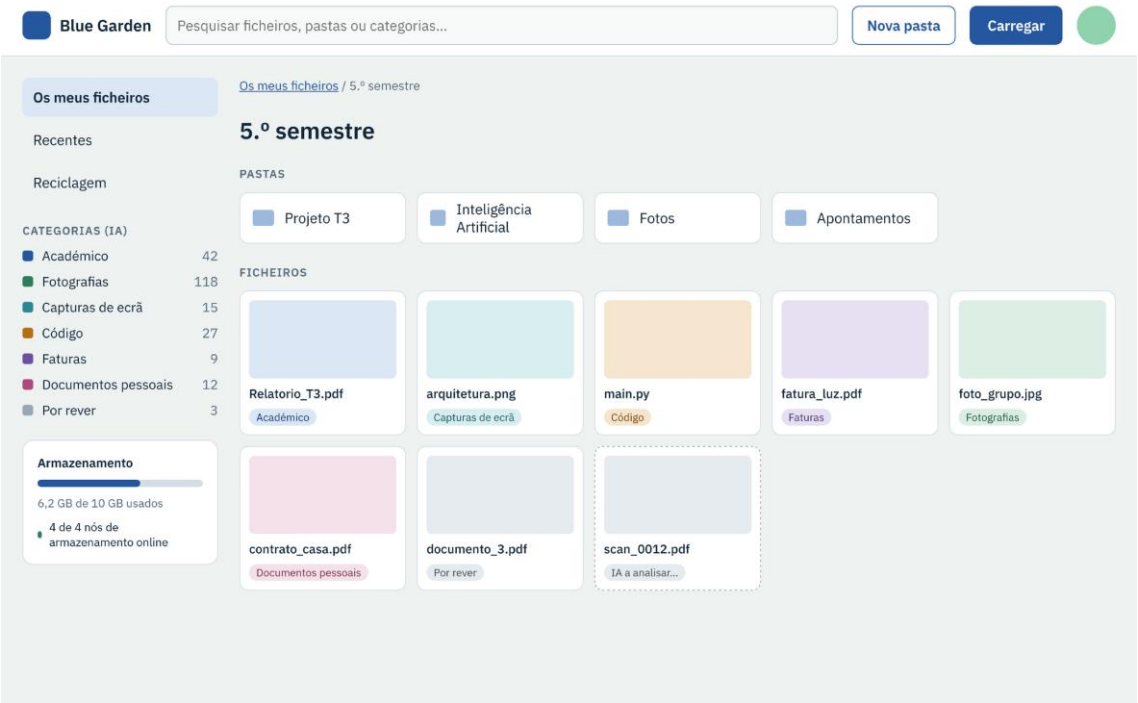


Figura 7. Mockup da página principal, com as categorias atribuídas pela IA

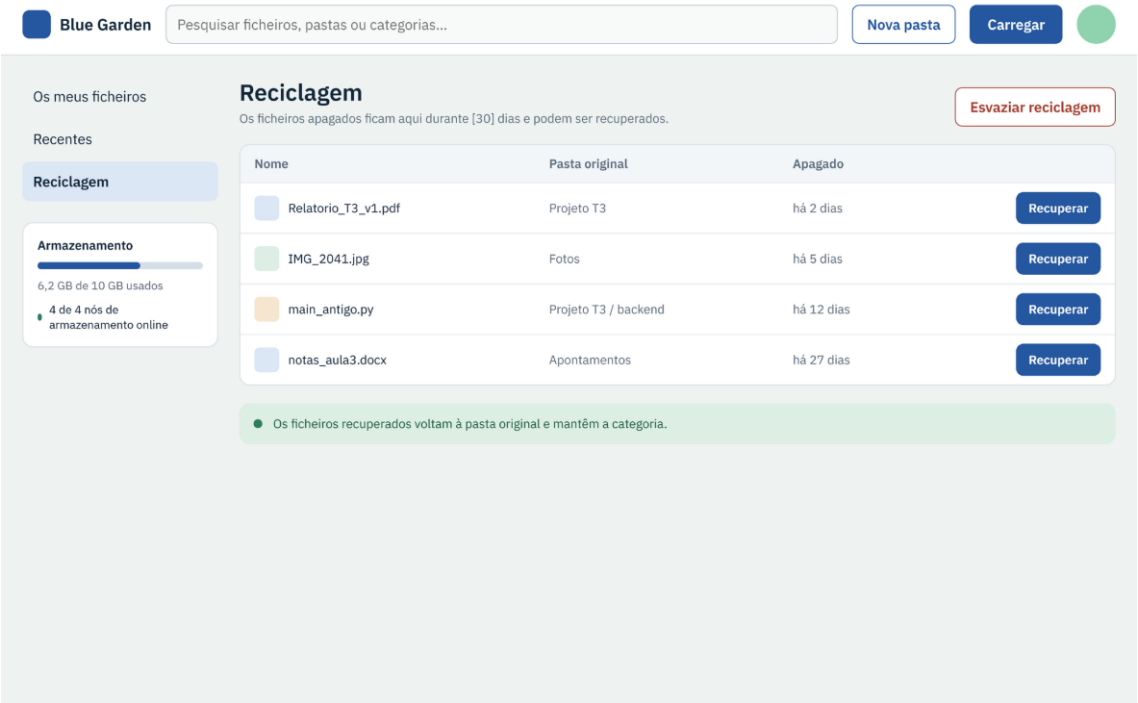


Figura 8. Mockup da reciclagem

Ficheiros técnicos

- **Arquitetura do sistema:** Figura 1.
- **Diagramas UML:** Figuras 2, 3 e 4.
- **Estrutura da base de dados:** Figura 3 (cada classe é uma coleção no MongoDB).
- **Capturas da aplicação:** mockups nas Figuras 6 a 8; capturas reais nas próximas fases.

- **Repositório GitHub:** https://github.com/mimofixe/BlueGarden

Anexos

Anexo A. Requisitos funcionais

ID	Descrição
RF01	O utilizador pode criar conta e fazer login
RF02	O utilizador pode fazer upload e download de ficheiros
RF03	O utilizador pode criar, renomear e apagar pastas
RF04	O utilizador pode mover ficheiros entre pastas
RF05	O utilizador pode apagar ficheiros e recuperá-los da reciclagem
RF06	O utilizador pode pesquisar ficheiros por nome
RF07	O sistema categoriza automaticamente cada ficheiro novo
RF08	O sistema mostra a categoria e o grau de confiança de cada ficheiro
RF09	O utilizador pode corrigir manualmente a categoria
RF10	O utilizador pode filtrar os ficheiros por categoria

Anexo B. Requisitos não funcionais

Requisito	Descrição
RNF01 Disponibilidade	O sistema continua a funcionar se cair um backend, um nó de armazenamento ou um nó da base de dados.
RNF02 Durabilidade	Nenhum ficheiro já confirmado se perde quando um nó de armazenamento falha.
RNF03 Tolerância a falhas da IA	Se o serviço de IA falhar, os uploads continuam e os ficheiros ficam em fila até ele voltar.
RNF04 Segurança	Comunicação por HTTPS, passwords guardadas em hash e ficheiros cifrados antes de serem armazenados.
RNF05 Privacidade	Cada utilizador só acede aos seus ficheiros. A análise da IA corre nos nossos servidores, sem serviços externos.
RNF06 Desempenho	A categorização acontece em segundo plano, sem atrasar o upload.
RNF07 Usabilidade	Interface web simples, que funciona em qualquer browser moderno e em qualquer dispositivo.
RNF08 Escalabilidade	É possível acrescentar instâncias do backend ou nós de armazenamento sem alterar o sistema.
RNF09 Fiabilidade da IA	Cada decisão tem um grau de confiança. Abaixo do limiar, o ficheiro vai para «Por rever».
RNF10 Manutenibilidade	Código versionado no GitHub. O sistema arranca com um único comando (Docker Compose).

Requisito	Descrição
RNF11 Nomes dos ficheiros	O sistema sugere um nome mais claro para cada ficheiro, com base no conteúdo.